created_at	row_id
2023-12-01	2
2023-12-02	1
2023-12-03	3

`row_id` refers to a row in the `transactions` table. It's an internal ID, not our primary key. Using this index all that MySQL can do is to filter rows between 2023-10-01 and 2024-01-11. Which results in roughly 94k rows. And then it goes through them reads the the rows from your disk and filters the final 10k rows based on the `user_id`.

If I delete the `order by` and run `explain` again this is the result:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	transactions	NULL	ref	user_id,created_at	user_id	8	con…	18560	50.00	Using where

Now it uses the `user_id` index and scans only 18k rows.

The big conclusion so far is this: **database indices do not exist in a vacuum.** They are specific to queries!

There's no thing such as "Hey I'll just add an index to the created_at column because you know... that's my default behavior as a developer. An index is always a good thing, isn't it?"

In this specific case, if I delete the created_at index I'll get better results!

You want to add indices based on the nature of your queries and features. For example, in this demo application most of the where expressions use both the `user_id` and the `created_at` columns as well.

Let's try a composite index!

```
CREATE INDEX user_id_created_at_idx ON transactions (user_id, created_at);
```

Let's run explain:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	transactions	NULL	range	user_id_created_a…	user_id_created_at_idx	13	NULL	18962	100.00	Using index condition

`range` type using the composite index scanning 18k rows. Check out Telescope:

